

Sovereign OS: An Open-Source, Agentic, Locally-Graphed Prototype of the Human Context Protocol

Sovereign OS Project

July 2026

Abstract

The Human Context Protocol (HCP) formalizes a user-owned, interoperable, consent-scoped preference layer that AI systems must query rather than silently absorb [1]. Its expanded specification, HCP-3, adds explicit CRUD semantics and transparent audit logging to that layer [2], and a March 2026 Stanford HAI / Gates Foundation / AWS / Mozilla Foundation workshop report decomposes a production-grade version of the same protocol into five concrete architectural components: a Memory Store, a Policy & Authorization Service, a Context Router, an Audit & Provenance Subsystem, and an Interoperability Interface [3]. Taken together, this body of work is considerably more architecturally complete than a single position paper, and it narrows the space of legitimate original contributions available to any system built on top of it. **Sovereign OS (SOS)** is proposed here as an **open-source, agentic, locally-graphed prototype implementation** of the Human Context Protocol / Model Context Protocol (HCP-MCP) stack that the HCP research program has specified but not yet shipped in production form. Three properties distinguish the proposed prototype from HCP’s own reference implementation and from the workshop’s architectural blueprint: (1) an **open-source** release under a permissive license, where HCP’s own reference implementation remains a public but unlicensed repository with no tagged releases, and the workshop report is specification only, with no accompanying codebase; (2) working **application and task assistance** – agentic delegation of real user tasks to sub-agents and third-party tools operating under the protocol’s consent and audit guarantees, rather than a preference lookup service; and (3) a **local, on-device memory graph** that satisfies HCP-3’s Encapsulation attribute using a portable graph representation, directly answering the workshop’s own call for on-device, tiered execution at the edge [3]. Sovereign OS’s founding premise is that personal computing history is a repeating cycle of centralization and reclamation, and that AI memory is the latest and highest-stakes round of that cycle.

Contents

1	Introduction	1
1.1	A Recurring Cycle: Centralization, Reclamation, and the Stakes of Memory	1
1.2	Positioning Statement	2
2	The Human Context Protocol Research Program	3
2.1	HCP: The Original Position Paper	3
2.2	HCP-3: CRUD Semantics, Audit, and an Expanded Author List	3
2.3	The Global Memory Workshop: A Five-Component Reference Architecture	4
2.4	What the HCP Research Program Leaves Open	5

3	Memory Architectures for Personal AI	5
3.1	Why Statelessness Forces an External Memory Architecture	5
3.2	Architectural Families	5
4	Sovereignty and User-Controlled Memory in Practice	7
4.1	Personal Data Stores: Solid and Inrupt	7
4.2	Local-First and On-Device Memory	7
4.3	How Commercial Products Expose or Withhold User Control	8
5	Delegated Reasoning and Agentic Orchestration	8
5.1	Tool Use and the Model Context Protocol	8
5.2	Agent-to-Agent Delegation	9
5.3	Task Decomposition and Human-in-the-Loop Trust Boundaries	9
6	Sovereign OS: A Prototype Implementation of the HCP-MCP Stack	9
6.1	Scope of the Contribution	9
6.2	Open-Source: A Community-Auditable Reference Implementation	9
6.3	Local Memory Graphing: Satisfying Encapsulation On-Device	10
6.4	Application and Task Assistance: Agentic Delegation Under Protocol Consent	10
6.5	Data Flow	11
7	Privacy, Security, and Governance	12
7.1	Regulatory Implications: GDPR and the Right to Erasure	12
7.2	Threat Model: Memory Poisoning	12
7.3	Threat Model: MCP Tool Poisoning	12
8	Comparative Positioning	13
9	Challenges and Future Directions	14
10	Conclusion	15

1 Introduction

1.1 A Recurring Cycle: Centralization, Reclamation, and the Stakes of Memory

The history of personal computing is a repeating argument about where state lives. Mainframe timesharing centralized computation inside institutions and rented cycles back to users by the minute. The personal computer revolution of the late 1970s and 1980s inverted that default: computation, storage, and control moved onto a machine an individual owned outright. Cloud computing and software-as-a-service reversed the inversion a second time, moving files, identity, and increasingly cognition itself back into infrastructure controlled by a small number of vendors. Tim Berners-Lee’s Solid project is an explicit attempt to run the reclamation half of that cycle again for personal data specifically, built on the premise that personal information should be “centred around people, instead of around apps” [12]. Solid’s Pods are personal online data stores where users “store information once and decide who can access what” [13], governed by Inrupt’s “fine-grained access controls” set by the owner rather than the application requesting access [14]. Security technologist Bruce Schneier summarized the model plainly: “Your data lives in a pod that is controlled by you . . . You authorize granular access to that pod to whoever you want” [15]. Real deployments already exist at civic

scale: Flanders’ data-utility company Athumi piloted a Solid-based recruitment application built with staffing firm Randstad and planned to scale it to 30,000 users during 2024, with Athumi’s chief executive describing the underlying shift as one where “the data subject becomes the data controller”; separately, five Belgian hospitals began storing hospital-visit records inside patients’ own Pods rather than a hospital-controlled database [12].

Personal AI memory is where this cycle reaches its highest stakes yet. A model that remembers a user’s health conditions, relationships, finances, and unfinished projects across sessions is not a convenience feature – it is a second copy of a person’s cognitive context, and whoever controls its storage, its access policy, and its export path controls a durable share of that person’s relationship with every AI system they will ever use. Sovereign OS takes its name, and its founding argument, from this history: that the correct default for personal AI memory is the personal-computer default, not the cloud-timesharing default, and that the recent flourishing of academic and institutional work formalizing exactly this position – discussed at length below – is evidence that the argument has moved from a philosophical position to an engineering research program with a name, a standards body of authors, and a growing set of prototypes.

1.2 Positioning Statement

HCP’s expanded specification, HCP-3, and a March 2026 multi-institution workshop report on the same protocol family establish that orchestration, delegated tool-execution, and audit are named, specified components of the protocol rather than gaps left open by it. HCP-3 already specifies Create/Read/Update/Delete (CRUD) actions over the protocol’s data store, exposed via REST or Model Context Protocol (MCP) tools, with transparent audit logging of agent access to data over time (Section 2.2) [2]. The workshop report goes further, decomposing a production HCP deployment into five named components – Memory Store, Policy & Authorization Service, Context Router / Memory Manager, Audit & Provenance Subsystem, and Interoperability Interface – with the Interoperability Interface itself defined around add, search, update, and delete operations [3].

The claim this paper defends is narrow and precisely scoped: HCP’s own research program, across three publications spanning May 2025 to March 2026, has moved from position paper to architectural blueprint, but it has not yet produced (a) an open-source, community-auditable implementation of that blueprint; (b) a working local, on-device *memory graph* that satisfies the protocol’s Encapsulation attribute without depending on cloud infrastructure; or (c) a demonstrated agentic delegation layer that lets the protocol’s consent and audit machinery actually authorize and record real third-party task execution on a user’s behalf, as opposed to preference lookups. Sovereign OS is proposed as exactly that: an open-source, agentic, locally-graphed *prototype* of the HCP-MCP stack, built as a reference implementation in the sense HCP’s own authors invite from the outset (Section 2.1) – one instance among the open set of systems the protocol’s design attributes permit.

2 The Human Context Protocol Research Program

2.1 HCP: The Original Position Paper

The Human Context Protocol was introduced by Anand V. Shah and Michiel A. Bakker (MIT), Tobin South (MIT), Talfan Evans (Persona Labs), Hannah Rose Kirk (University of Oxford / UK AI Security Institute), Andrew Trask (OpenMined), and E. Glen Weyl (Microsoft Research), who argue that “robust AI personalization requires a Human Context Protocol (HCP): a user-owned, secure,

and interoperable preference layer that grants individuals granular, revocable control over how their data steers AI systems” [1]. The paper specifies four design attributes any conforming system must satisfy: **Interoperable** (facilitated by “open, well-supported, and existing communication protocols”); **Sufficiently Representative** (a data model “at least as expressive as state-of-the-art techniques in preference capture”); **Scopable Sharing** (“fine-grained, revocable, and editable control over what preference information is shared and with whom . . . a user should be able to share culinary preferences with a recipe generator without exposing mental health information”); and **Secure** (“preferences must be secured at rest and in transit, with strong authentication and authorization mechanisms”) [1].

The authors built and shipped a proof-of-concept: a web application in which users manage preferences and third-party access, backed by an integrated MCP server that enforces “user authentication and granular authorization for distinct preference categories,” with preferences “stored as categorized textual data in PostgreSQL” and a GPT-4o-based “HCP LLM” that “ingests only authorized preference categories” per query [1]. The reference implementation is publicly available [4]. That repository (github.com/tobinsouth/hcp-demo), maintained by co-author Tobin South as a Stanford Digital Economy Lab / Consumer Reports collaboration, is in practice a Next.js application that stages the protocol’s story across three progressive scenes – Human Context, Grant of Authority, and Applications – rather than exposing a runnable backend a third party could deploy and audit end to end. As of mid-2026 it carries no license file and no tagged release, a discrepancy with the original paper’s own text, which describes this same prototype as “open-source” [1, 4]. This is not a criticism of the demo’s communicative purpose, but it is the first of several concrete gaps between HCP’s stated intentions and its shipped artifacts that motivate a genuinely open-source implementation, taken up directly in Section 8. Critically, the original paper is explicit that the prototype is minimal and that specification, not implementation, is its contribution: “this paper does not prescribe a definitive implementation for HCP; any system fulfilling the aforementioned design attributes would be suitable” [1].

2.2 HCP-3: CRUD Semantics, Audit, and an Expanded Author List

The protocol’s second published iteration, retitled “Robust AI Personalization Controls: The Human Context Protocol” and co-authored with Jiaxin Pei (Stanford), expands the specification in ways that materially change what a downstream implementer can claim as novel [2]. Two changes matter most for this paper’s positioning. First, the four design attributes are renamed and one is substantively redefined: **Interoperability**, **Encapsulation**, **Control**, and **Security**. Encapsulation explicitly broadens the data model beyond categorized text, stating that the representation “should leverage text, graph, or vector-based representations,” provided it remains portable across applications [2]. This is a direct, protocol-level endorsement of graph-based memory representations as HCP-compliant – a fact this paper leans on heavily in Section 6, since it means a local memory graph is not merely compatible with HCP by coincidence, but satisfies one of the protocol’s own four named attributes more fully than the original PostgreSQL key-value prototype did.

Second, HCP-3 specifies CRUD actions – Create, Read, Update, and Delete – executable over the protocol’s data store via REST endpoints or MCP tools, and states that “transparent audit logs allow users to review exactly which agents accessed which data and when” [2]. An audit layer is therefore a named, specified part of the protocol itself, not an unaddressed gap. What HCP-3 does not establish is a *working, open-source, community-auditable implementation* of that audit layer; it remains a specification accompanied by a single reference prototype that is publicly visible but carries no open-source license, no tagged releases, and no independent verification in production use

[4].

2.3 The Global Memory Workshop: A Five-Component Reference Architecture

The most consequential document for scoping Sovereign OS’s contribution is not a paper by the original HCP authors at all, but the final report of a March 11–12, 2026 workshop convened by Stanford HAI, the Gates Foundation, AWS, the Mozilla Foundation, and Slalom, titled “The Future of Personal AI: Portable and Persistent Personal Memory through a Unified Human Context Protocol” [3]. Its listed researchers – Sandy Pentland, Tobin South, Dazza Greenwood, Jiaxin Pei, Zexue He, and Ben Moskowitz – overlap substantially with the HCP authorship, indicating this is a continuation of the same research program rather than a competing effort, hosted on Stanford’s Digital Economy Lab project page for HCP [3].

The workshop report decomposes a production-grade Unified Human Context Protocol into five named architectural components:

1. **Memory Store** – the persistence layer for personal context, decoupled from any single application.
2. **Policy & Authorization Service** – the component enforcing scoped, revocable access, generalizing HCP’s Control attribute.
3. **Context Router / Memory Manager** – the retrieval and routing layer that decides what context reaches which requester.
4. **Audit & Provenance Subsystem** – a dedicated component for logging and reconstructing who accessed what, when, and why.
5. **Interoperability Interface** – exposed through add, search, update, and delete operations, generalizing HCP’s original Interoperability attribute into a concrete API surface [3].

Orchestration, audit, and interoperable execution – capabilities that a reading of the original 2025 HCP position paper alone might suggest are unaddressed – each map onto a named component in this five-part architecture. The protocol ecosystem around HCP is, as specification, considerably more built out than the original position paper alone would indicate.

That said, the workshop report is equally explicit about what remains unresolved, and its own language is the strongest available textual anchor for the gap Sovereign OS is positioned to fill. A dedicated section on “On-Device and Small Model Constraints” calls for “distillation and fine-tuning toward memory tasks,” “hybrid pipelines combining small models with deterministic components,” and “tiered execution” in which local, on-device models handle sensitive operations rather than deferring them to cloud infrastructure [3]. None of these are described as solved – they are described as requirements for future work. The report also repeatedly frames HCP’s own reference implementation as “an early step” and a proof-of-concept rather than a production system [1, 3], and it frames the core systemic risk the protocol is designed to counter as centralization of personal context “inside a handful of frontier model providers” [3] – language that directly reinforces this paper’s sovereignty framing in Section 1.

2.4 What the HCP Research Program Leaves Open

Synthesizing the three publications, three concrete gaps remain open as of mid-2026, and they define the scope of Sovereign OS’s proposed contribution for the remainder of this paper:

1. **No open-source, production-oriented implementation.** HCP’s own prototype is a single research demo built to validate the specification, not a community-maintained, auditable,

redistributable codebase intended for other developers to run and extend in production.

2. **No working local memory graph.** The workshop’s own call for on-device, small-model, tiered execution is unaddressed by any of the three publications’ reference implementations, all of which store preferences in a centralized PostgreSQL instance rather than a local, portable graph structure.
3. **No demonstrated agentic delegation layer.** While HCP-3 specifies CRUD actions and audit logging over the memory store itself, none of the three publications demonstrate a sub-agent or third-party tool actually executing a delegated task under the protocol’s consent and audit guarantees – the protocol governs access to context, not the downstream use of that context to act.

These three gaps map directly onto the three properties this paper’s title and abstract emphasize: open-source, application/task assistance, and local memory graphing. The remainder of this paper surveys the broader technical landscape each gap sits within, then specifies Sovereign OS as a prototype addressing all three simultaneously.

3 Memory Architectures for Personal AI

3.1 Why Statelessness Forces an External Memory Architecture

Large language models are fundamentally stateless: each inference draws only on the tokens placed in a bounded context window, and nothing persists once that window closes. The MemGPT paper frames the problem directly, observing that LLMs “are constrained by limited context windows, hindering their utility in tasks like extended conversations and document analysis” [5]. The Mem0 authors make the same point about long-horizon interaction specifically: “fixed context windows pose fundamental challenges for maintaining consistency over prolonged multi-session dialogues” [6].

Simply enlarging the window does not resolve this. Anthropic’s engineering guidance describes “context rot,” whereby “as the number of tokens in the context window increases, the model’s ability to accurately recall information from that context decreases,” a property that “emerges across all models” because the transformer architecture’s pairwise attention computation is “stretched thin” at long context lengths [7]. Context must therefore be treated “as a finite resource with diminishing marginal returns,” the premise behind the discipline of context engineering: “the set of strategies for curating and maintaining the optimal set of tokens . . . during LLM inference” [7].

3.2 Architectural Families

Retrieval-augmented generation (RAG) embeds text into vectors, stores them in a vector database, and retrieves the most semantically similar chunks at query time. It is simple and cheap to write, but stale and imprecise when embeddings blur distinct facts, and it cannot represent evolving relationships or temporal validity – a direct challenge to HCP’s Sufficiently Representative / Encapsulation attribute if used naively as a preference-capture mechanism.

Hierarchical, OS-style paging, introduced by MemGPT (now developed as Letta), draws “inspiration from hierarchical memory systems in traditional operating systems” and pages information between an in-context working set and an external store via model-issued function calls [5]. This is the architectural metaphor Sovereign OS borrows for its own “Operating System” framing.

Memory streams with reflection, from the Stanford Generative Agents work, record “a comprehensive list of the agent’s experiences” in natural language and periodically synthesize higher-level

insights through a reflection process that fires when accumulated importance scores cross a threshold [8]. Retrieval combines recency, importance, and semantic relevance as a weighted score [8].

Temporal knowledge graphs, exemplified by Zep, organize memory into episode, entity, and community subgraphs with a bi-temporal model tracking both when a fact was true in the world and when it was ingested, enabling “edge invalidation” – marking superseded facts outdated while preserving history – rather than deletion [9]. Zep’s production benchmark page reports 94.7% accuracy on LoCoMo and 90.2% on LongMemEval [10], though these figures are vendor self-reported; a March 2026 comparison published by Vectorize – itself a vendor of a competing memory product, Hindsight, which the same comparison recommends evaluating – reported Zep at only 63.8% and Mem0 at 49.0% on LongMemEval using GPT-4o, well below either vendor’s own claims [11]. Neither side of this comparison is a neutral source: the divergence between Zep’s and Mem0’s self-reported figures and a rival vendor’s unfavorable re-measurement of both is itself a standing caution against treating any single vendor-published memory benchmark as load-bearing evidence for an architecture decision.

Managed extraction pipelines, exemplified by Mem0 and its graph-augmented variant Mem0^g, “dynamically extract, consolidate, and retrieve salient information,” reporting “26% relative improvements in the LLM-as-a-Judge metric over OpenAI” on the LOCOMO benchmark, with the graph variant scoring roughly two points higher again [6] – a figure self-reported by Mem0’s own co-founders (the paper’s listed authors include Mem0’s CEO and CTO) rather than an independently verified result, the same caveat that applies to the Zep figures discussed next.

Approach	Strengths	Weaknesses / risks
RAG / vector store	Simple, cheap writes, scalable	Staleness; retrieval imprecision; no relational or temporal modeling [6]
Hierarchical paging (MemGPT/Letta)	Extends effective context; self-managed	Added control-flow complexity [5]
Memory stream + reflection (Generative Agents)	Captures recency/importance/relevance; compresses via reflection	Hand-tuned weights; reflection cost [8]
Temporal knowledge graph (Zep)	Relational reasoning; fact invalidation; non-lossy history	Graph construction overhead; vendor benchmarks unverified independently [9, 11]
Managed extraction (Mem0/Mem0 ^g)	Strong benchmark accuracy; large token savings	Extraction-LLM dependence; sensitive-data leakage risk; founder-authored benchmark, unverified independently [6]
HCP preference store	Simple; MCP-native; scoped sharing; revocable	Narrow scope (preferences, not general graph memory); centralized PostgreSQL [1]

Table 1: Memory-architecture families relevant to a personal AI implementation.

None of the general-purpose memory systems above were designed against HCP’s Scopable Sharing / Control attribute – fine-grained, revocable, per-recipient disclosure – and none, including HCP’s own reference prototype, implement the local graph representation HCP-3’s Encapsulation attribute explicitly permits [2]. This is the first of the three concrete openings Sovereign OS is positioned to occupy.

4 Sovereignty and User-Controlled Memory in Practice

4.1 Personal Data Stores: Solid and Inrupt

The most developed standards effort for user-owned data remains the W3C Solid project. Inrupt’s implementation uses per-resource Access Control Policies and verifiable credentials, such that “if a resource has no Policies that apply to it, the resource is inaccessible” by default [14]. HCP’s own authors situate their work in this lineage explicitly, citing “earlier work in personal data stores” as the foundation they extend to “the scale and stakes of modern generative AI” [1]. Solid demonstrates that standards-based personal data sovereignty is deployable at civic scale, but its application-developer ecosystem remains comparatively small relative to the AI personalization space HCP targets.

4.2 Local-First and On-Device Memory

A parallel movement builds memory that never leaves the user’s hardware, and by 2026 it has moved well past forum speculation into shipped, benchmarked systems. Stanford’s Hazy Research and Scaling Intelligence Lab, together with Lambda Labs, released OpenJarvis in mid-2026: an Apache-2.0, MCP-native framework that runs inference, agent state, and interchangeable memory backends entirely on-device, landing within 3.2 percentage points of frontier cloud-model accuracy across eight benchmarks while cutting marginal cost roughly 800-fold and end-to-end latency roughly four-fold [16]. Apple Intelligence embodies the on-device model at consumer scale: its “cornerstone . . . is on-device processing, so it is aware of your personal information without collecting your personal information,” and includes an “on-device semantic index that can organize personal information from across apps” [18], with a Private Cloud Compute tier for heavier workloads that processes “only the data that is relevant to the task” and is not retained [17]. Independent academic verification of such vendor claims remains rare and technically demanding: the first published reverse-engineering study of Private Cloud Compute had to build its own tooling just to query the system outside Apple’s intended interfaces, noting “there are no reproducible builds, and there are no symbols within those binaries,” and found the underlying proprietary model’s independently benchmarked accuracy comparable to, but below, Apple’s own published figures [19] – a reminder that sovereignty claims require independent verification rather than vendor assertion, a standard this paper applies to its own claims below. Neither OpenJarvis nor Apple Intelligence implements HCP’s specific consent, revocation, or scoped-sharing semantics: OpenJarvis is a general-purpose local-agent framework with no per-recipient access control over what a connected tool may read from memory, and Apple Intelligence exposes no interoperable protocol for third parties to request scoped access at all. The gap this paper positions Sovereign OS to fill is therefore not “local, on-device AI” – OpenJarvis already demonstrates that convincingly – but the specific combination of local execution with HCP’s own consent-scoped, auditable interoperability layer, developed at length in Section 8.

4.3 How Commercial Products Expose or Withhold User Control

Commercial assistants have converged on a common structural pattern – “compressed long-term memory paired with raw working memory” [21] – while diverging sharply in transparency and portability.

User control has improved across the industry – deletion, project scoping, and temporary modes are now common – but standards-based, full-export portability remains weak. Only Claude documents an import path for other vendors’ exports [24], and no fetched vendor source documents a full,

Product	Storage	User control	Portability
ChatGPT	Cloud	Manage, per-item delete, clear all [22]	Not documented in fetched sources
Claude	Cloud	Per-item + clear all, Temporary Chat [23]	Import tool for other chatbots’ exports [24]
Gemini	Cloud	Toggle, delete, Temporary Chats [25]	Not documented in fetched sources
Copilot / Recall	Local (encrypted)	Exclude apps/sites, delete ranges, uninstall [26]	Local DB on device
Perplexity	Cloud (encrypted)	Per-item + bulk clear; 30-day cleared log [27]	Not documented in fetched sources
Apple Intelligence	On-device + PCC	Per-app iCloud toggles; exportable PCC report [17]	Local device storage
HCP prototype	Cloud (PostgreSQL)	Web UI; per-category authorization [1]	Interoperable by design (MCP-native)

Table 2: Storage, control, and portability across commercial personal-AI memory products and the HCP reference prototype.

standards-based export of its memory store. HCP is the first academically formalized, MCP-native, and demonstrated attempt to close that gap at the protocol level, which is why it is the correct anchor for this paper rather than a competitor to differentiate against.

5 Delegated Reasoning and Agentic Orchestration

5.1 Tool Use and the Model Context Protocol

Anthropic’s Model Context Protocol (MCP), announced November 25, 2024, standardizes the boundary between a model and external tools as “a universal, open standard for connecting AI systems with data sources, replacing fragmented integrations with a single protocol” [28]. MCP servers expose Tools, Resources, and Prompts over JSON-RPC 2.0, and the specification embeds a consent model in which “every tool call goes through user approval” [29] — framed as a SHOULD-level recommendation rather than a mandatory guarantee, leaving the precise interaction model to the host implementation. Adopted subsequently by OpenAI, which added MCP support across its Agents SDK and ChatGPT products after its CEO said “people love MCP and we are excited to add support across our products” [30], and by Google DeepMind, which added native MCP support to the Gemini API and SDK [31], MCP is, a year and a half after its release, “the closest thing the AI industry has to an actual standard” [29]. This is the same protocol family HCP’s own reference prototype uses as its interoperability transport [1], and the same family the workshop report’s Interoperability Interface is built around [3] — which is precisely why an HCP-compliant prototype with an agentic delegation layer can be built as an extension of the same wire protocol HCP already committed to, rather than a competing transport.

5.2 Agent-to-Agent Delegation

Google’s Agent2Agent (A2A) protocol, introduced April 2025 and later contributed to the Linux Foundation, standardizes how “different, independent AI agents communicate and collaborate with each other” [32]. A2A advertises agent capabilities via signed “Agent Cards” and, critically, “allow[s]

agents to collaborate without needing to share internal memory, proprietary logic, or specific tool implementations” [34], preserving opacity between a delegating agent and the agents or tools it delegates to. A2A and MCP are explicitly complementary: A2A “complements Anthropic’s Model Context Protocol (MCP), which provides helpful tools and context to agents” [32]. HCP’s own paper gestures at A2A as a candidate integration path but, consistent with the gaps identified in Section 2.4, does not specify how a preference layer would mediate or audit cross-agent delegation in practice [1].

5.3 Task Decomposition and Human-in-the-Loop Trust Boundaries

The prevailing orchestration pattern dispatches specialized sub-agents that “handle focused tasks with clean context windows” and return “only a condensed, distilled summary of its work,” preserving a clear separation of concerns between exploration and synthesis [7]. Both major delegation protocols push consent decisions to a human: MCP’s specification states that when servers request completions via sampling, “there SHOULD always be a human in the loop with the ability to deny sampling requests,” and that applications “SHOULD” present confirmation prompts for tool calls, while explicitly noting that “the protocol itself does not mandate any specific user interaction model” [29]. This human-in-the-loop guidance is the primary control preventing a delegated sub-agent or tool from acting beyond its authorized scope – and, as Section 7 shows, it is necessary but not sufficient on its own.

6 Sovereign OS: A Prototype Implementation of the HCP-MCP Stack

6.1 Scope of the Contribution

Sections 2 through 5 establish that the HCP research program already specifies interoperability, representational encapsulation (including graph-based representations), scoped consent, and audit logging as protocol-level requirements, and that the broader field has separately developed strong memory-architecture and delegation-protocol building blocks. Sovereign OS’s proposed contribution is therefore integrative and implementation-focused rather than architectural invention: it is an attempt to be the first *open-source, agentic, locally-graphed* system that instantiates the full HCP-MCP stack end to end, addressing the three concrete gaps identified in Section 2.4. Each of the three qualifiers corresponds to a specific, falsifiable design commitment.

6.2 Open-Source: A Community-Auditable Reference Implementation

HCP’s own authors invite exactly this, having stated from the outset that the protocol does not prescribe a definitive implementation and that any system satisfying its design attributes is suitable (Section 2.1) [1]. Their own reference implementation, while publicly visible on GitHub, carries no open-source license file, no tagged releases, and no visible external contributor activity beyond its original authors [4], and none of the three publications surveyed in Section 2 describe a community governance model, a public issue tracker, or a versioned release process for their prototype. Sovereign OS is proposed as a permissively licensed, publicly maintained codebase implementing the workshop report’s five-component architecture – Memory Store, Policy & Authorization Service, Context Router, Audit & Provenance Subsystem, and Interoperability Interface [3] – such that the audit and provenance claims made about the system are independently verifiable by any developer who chooses to read the code, rather than resting on the reputation of a single, unlicensed research

prototype. This is a meaningful sovereignty claim in its own right: a protocol whose only reference implementation lacks a license, a release process, and community governance has not actually achieved the decentralization of control its specification calls for, no matter how carefully that specification is written.

6.3 Local Memory Graphing: Satisfying Encapsulation On-Device

HCP-3’s Encapsulation attribute permits graph-based data representations, provided they remain portable (Section 2.2) [2]. None of the three publications’ reference implementations exercise the graph option; all store preferences in a centralized PostgreSQL instance. The workshop’s on-device and small-model constraints, discussed in Section 2.3, call for exactly this kind of local, tiered execution [3] – a call this paper reads as a direct invitation to the local-graph design Sovereign OS proposes.

A concrete existence proof that local graph-based memory is already low-risk, native infrastructure within HCP’s own protocol family – rather than an analogy borrowed from an unrelated domain – comes from the Model Context Protocol’s own reference server suite. The official `@modelcontextprotocol/server-memory` package implements “a basic implementation of persistent memory using a local knowledge graph” that “lets Claude remember information about the user across chats,” storing entities, relations, and observations as a JSON graph on the local filesystem under an MIT license [20]. Because HCP already commits to MCP as its own interoperability transport (Section 5.1), this reference server demonstrates that a local, on-device knowledge graph is not merely compatible with HCP by coincidence, but is already shipping, maintained infrastructure inside the exact protocol stack HCP’s reference implementation depends on. OpenJarvis’s “interchangeable memory backends” primitive, discussed in Section 3.2, is a second, independent confirmation that production-grade local memory representations now run at the framework level rather than only in research demos [16]. Neither system, however, implements any of HCP-3’s Encapsulation, Control, or Security semantics: the MCP reference memory server has no notion of per-category consent, revocation, or audit – any client with tool access can read or write the entire graph – and OpenJarvis’s memory backends are similarly ungated with respect to scoped sharing. Sovereign OS’s local memory graph is proposed to be exactly that missing connective layer: an on-device graph store built on the same MCP transport already native to both HCP and the wider local-agent ecosystem, but gated by HCP-3’s Encapsulation-compliant scoped consent and exposed through the workshop’s own Interoperability Interface (add/search/update/delete), rather than a closed, siloed application feature or an ungated shared graph.

6.4 Application and Task Assistance: Agentic Delegation Under Protocol Consent

The third gap is the least resolved of the three. HCP-3 specifies CRUD access to a memory store and audit logging of who accessed what and when [2]; the workshop report names a Policy & Authorization Service and an Audit & Provenance Subsystem [3]. None of these publications demonstrate a downstream agent actually *using* authorized context to execute a real task – booking, drafting, purchasing, scheduling – under the protocol’s consent and audit guarantees. Sovereign OS’s agentic layer is proposed to close this specific gap by implementing task-oriented sub-agent delegation on top of, not alongside, the protocol’s own authorization and audit primitives: every sub-agent dispatch and third-party tool call reads its authorized context scope from the Policy & Authorization Service, executes via MCP tool calls subject to the specification’s human-in-the-loop approval guidance [29], and writes a structured record to the Audit & Provenance Subsystem rather

than to a separate, SOS-specific logging system. This is a deliberately conservative design choice: it treats the workshop’s five-component architecture as the target interface to implement against, rather than proposing a sixth, competing component.

Workshop component [3]	HCP-3 / workshop specification status	Sovereign OS prototype role
Memory Store	Specified; reference implementation is centralized PostgreSQL [1]	Local, on-device graph store (Section 6.3)
Policy & Authorization Service	Specified conceptually; not demonstrated at delegation scale	Authorization scope consulted before every sub-agent dispatch (Section 6.4)
Context Router / Memory Manager	Specified conceptually	Just-in-time retrieval from the local graph
Audit & Provenance Subsystem	Specified; CRUD audit logging described in HCP-3 [2]	Single audit log spanning memory access <i>and</i> delegated task execution
Interoperability Interface	Specified as add/search/update/delete over MCP/REST [3]	Implemented as an open-source, versioned MCP server

Table 3: Mapping of the Global Memory Workshop’s five-component architecture to Sovereign OS’s proposed prototype role. Sovereign OS implements against, rather than beside, each specified component.

6.5 Data Flow

A user query enters the local agentic layer, which consults the Policy & Authorization Service for the scope of context it may read. The Context Router retrieves relevant nodes from the local memory graph, gated by that authorization. The agentic layer decomposes the task and dispatches sub-agents that call MCP tools – each call consent-gated per the protocol specification – and, where a remote agent is needed, hands off via A2A without exposing the local graph’s internal structure [34]. Results return as distilled summaries; new salient facts are written back into the local graph through the same authorization gate; and every read, write, dispatch, and tool call is recorded in the Audit & Provenance Subsystem, producing a single reconstructable record of “what an agent did on the user’s behalf,” regardless of which sub-agent or tool executed it.

7 Privacy, Security, and Governance

7.1 Regulatory Implications: GDPR and the Right to Erasure

GDPR Article 17’s right to erasure and Article 20’s right to portability sit in tension with LLM memory. “AI models do not store information in discrete entries,” so “once personal data is integrated into a model’s parameters, removal becomes nearly infeasible without costly retraining or experimental machine unlearning methods” [35]. The European Data Protection Board notes that satisfying the right to erasure “requires reversing the memorisation of personal data by the model,” with options ranging from full retraining to sharded retraining and parameter-level unlearning [36]. This problem largely dissolves for a well-designed memory *store* external to model weights: production vector databases now offer metadata-filtered bulk deletion explicitly marketed to “honor

Right to be Forgotten requests (GDPR) with one API or SDK call” [37], which in principle lets an operator tag embeddings with a `data_subject_id` and purge by filter rather than collecting IDs by hand. In practice this guarantee is weaker than it appears: recent work on HNSW-indexed vector stores shows that soft-deleted embeddings typically remain physically present in the on-disk index, and an embedding-inversion attack can reconstruct deleted personal identifiers and structured fields directly from the untouched vectors — 25.5% of person names and 46.4% of locations on its headline dataset, and up to 100% on narrow structured fields such as age and gender — even after a delete-by-ID or delete-by-metadata call has completed [38]. A local graph store with a literal, relationally-enforced `DELETE CASCADE` avoids this failure mode by construction: deleting a node removes its properties and every dependent edge in a single transaction at the storage layer, rather than leaving a soft-deleted, potentially recoverable index entry behind – a direct architectural argument for the local-graph design proposed in Section 6.3, and the same argument HCP-3’s Security attribute rests on, made here at the level of implementation mechanics rather than design principle [2].

7.2 Threat Model: Memory Poisoning

Persistent memory expands the attack surface. The MINJA (Memory Injection Attack) demonstrates that an adversary, “without assuming that the attacker can directly modify the memory bank,” can inject malicious records “by only interacting with the agent via queries and output observations,” using bridging steps that later cause harmful reasoning when a victim issues a related query [39]. Because “any user [can] influence agent memory,” this is a low-barrier, high-impact attack class [39], and it is not addressed by HCP’s published threat model, which stops at unauthorized preference disclosure rather than adversarial injection into the store itself [1, 2].

7.3 Threat Model: MCP Tool Poisoning

Delegated reasoning imports the tool layer’s vulnerabilities. Invariant Labs documents MCP Tool Poisoning as an indirect prompt injection in which “malicious instructions are embedded within MCP tool descriptions that are invisible to users but visible to AI models”; because “AI models see the complete tool descriptions ... while users typically only see simplified versions in their UI,” the model reads attacker instructions the human never sees [43]. Concrete 2025 CVEs illustrate the class: CurXecute (CVE-2025-54135, CVSSv3 8.5), a remote-code-execution flaw in the Cursor editor reachable via prompt injection when an MCP-connected tool is used to rewrite `mcp.json`, and MCPoison (CVE-2025-54136, CVSSv3 7.2), which abuses trust-by-name rather than trust-by-content in approved MCP server configurations to smuggle in silently-executed commands [41]. JFrog’s disclosure of CVE-2025-6515 (“Prompt Hijacking”) showed session-ID reuse could cause a model to “present the user with the attacker’s intended response, while dropping the response for the user’s original request” [42]. Recommended mitigations include version and hash pinning of MCP servers and tools, cross-server dataflow boundaries, and refusing to auto-approve tool calls [43]. Because Sovereign OS’s agentic layer is proposed to execute arbitrary third-party MCP tools on the user’s behalf (Section 6.4), this threat class is the primary attack surface its Audit & Provenance Subsystem and human-in-the-loop gate must defend against; it is not addressed by any of the three HCP publications, whose threat model does not extend past the memory store’s own access controls.

8 Comparative Positioning

Four concrete, evaluated systems bound the design space Sovereign OS occupies as of mid-2026. HCP’s own protocol family, not Sovereign OS, specifies interoperability and audit as named requirements; as Table 4 shows, Sovereign OS’s differentiation is confined to the open-source, local-graph, and demonstrated-delegation columns.

System	Protocol status	Open-source?	Local memory graph?	Demonstrated agentic delegation?
HCP / HCP-3	Specifies interoperability, encapsulation, control, security [1, 2]	No – public repo, unlicensed, no releases [4]	No – centralized PostgreSQL	Gestured at (MCP/A2A mentioned), not demonstrated
Global Memory Workshop architecture	Specifies five-component reference architecture [3]	No – report only, no codebase	No – calls for it as future work	No – specification only
HCP-Demo (reference impl.)	Illustrates HCP’s own consent-scoped model; narrative walkthrough, not a deployable backend [4]	No – public repo, no license file, no tagged releases [4]	No – Next.js narrative app, no graph store	No – no delegation layer
Open-Jarvis	No relation to HCP; general-purpose local-agent framework [16]	Yes – Apache 2.0, public releases [16]	No HCP-compliant graph – interchangeable, ungated memory backends [16]	Yes – on-device agent + MCP tool use, benchmark-evaluated [16]
Sovereign OS (proposed)	Prototype implementation of HCP-MCP stack	Yes – permissive license, public code-base	Yes – on-device graph, HCP-3 Encapsulation-compliant	Yes – sub-agent + MCP tool execution under protocol audit

Table 4: Comparative positioning as of mid-2026. Sovereign OS’s claim is confined to being open-source, locally graphed, and agentially demonstrated; interoperability and audit are specified by HCP’s own program.

HCP’s own research program has already specified the interoperability, encapsulation, control, and security/audit requirements a personal AI memory protocol needs, and the Global Memory Workshop has already decomposed that specification into five concrete components. The two closest existing artifacts sit on opposite sides of the gap Sovereign OS proposes to close. HCP-Demo, the protocol’s own reference implementation, demonstrates the consent-scoped preference model end to end but is a narrative walkthrough rather than a licensed, deployable, or extensible codebase, and implements neither a local memory graph nor agentic task delegation [4]. OpenJarvis, released independently of the HCP program in mid-2026, demonstrates that a fully local, open-source, MCP-native agentic stack with interchangeable memory backends can already match near-frontier cloud accuracy on real benchmarks at a fraction of the cost and latency [16] – but it implements no consent, revocation, audit, or scoped-sharing semantics over that memory at all; any tool with access

reads the whole store. Neither system is a competitor to Sovereign OS’s proposed contribution: HCP-Demo has the protocol’s semantics but not the open, local, agentic execution; OpenJarvis has the open, local, agentic execution but not the protocol’s semantics. No published system combines an open-source release, a local memory graph satisfying HCP-3’s Encapsulation attribute, and a demonstrated agentic delegation layer that is gated by HCP’s own consent, revocation, and audit machinery rather than left ungated. That specific combination – not local AI in general, and not agentic delegation in general, both of which already exist elsewhere – is the concrete, scoped, and currently unclaimed contribution this paper proposes Sovereign OS build, with HCP-Demo, the Global Memory Workshop’s specification, and OpenJarvis serving as its three most relevant baselines.

9 Challenges and Future Directions

Several open problems remain within this scope. **Memory consistency and forgetting** are unresolved in general: bi-temporal graphs with edge invalidation [9] handle changing facts more gracefully than flat RAG, but no consensus exists on when an agent should proactively forget, and reflection weights in memory-stream architectures remain hand-tuned rather than learned [8]. **Benchmark reliability** is a standing concern: the wide divergence between vendor-published memory-system accuracy figures, including a competing vendor’s own unfavorable re-measurement of its rivals [11], means any evaluation of a Sovereign OS prototype against LOCOMO, Deep Memory Retrieval, or LongMemEval must be run by a disinterested party rather than cited from any vendor’s claims, and none of these benchmarks yet measure delegation-audit completeness or poisoning resistance specifically [39, 40]. **Cross-device and cross-vendor interoperability** remains weak industry-wide despite HCP’s specification work: no fetched vendor source documents a full, standards-based export of its memory store, and portability today is largely limited to Claude’s import of other chatbots’ exports [24]. **On-device model capability** is the workshop’s own identified constraint: distillation, hybrid small-model/deterministic pipelines, and tiered execution are called for but not yet demonstrated at the fidelity cloud-hosted models achieve [3], and a Sovereign OS prototype inherits this constraint directly since its local graph and local agentic layer depend on it.

On institutional trajectory, the most consequential recent development is the speed at which the HCP research program has moved from a multi-author position paper spanning MIT, Persona Labs, Oxford, OpenMined, and Microsoft Research (May 2025) to a five-institution workshop with a named reference architecture convened by Stanford HAI, the Gates Foundation, AWS, the Mozilla Foundation, and Slalom (March 2026), in under a year [1, 3]. This is strong evidence that the protocol layer this paper’s founding sovereignty argument depends on is no longer a speculative proposal but an active, multi-institution standardization effort, alongside MCP’s continued adoption across OpenAI [30] and Google DeepMind [31] and A2A’s transfer to the Linux Foundation [33]. Sovereign OS’s proposed role in that trajectory is not to originate the protocol – that work is already well underway across multiple institutions – but to be the first open-source, agentic, locally-graphed system to actually run it.

10 Conclusion

Personal AI memory is the newest round of a decades-old cycle between centralized and personally controlled computing, and the Human Context Protocol research program – now spanning a

2025 position paper, an expanded 2026 specification with CRUD and audit semantics, and a five-institution workshop decomposing the protocol into a concrete reference architecture – has moved that cycle’s reclamation argument from philosophy to engineering with unusual speed. Sovereign OS’s contribution is the first proposed open-source, agentic, locally-graphed prototype of the protocol stack HCP’s own authors and the Global Memory Workshop have already specified and explicitly invited the community to build. The philosophical and historical argument for that project – that personal computing sovereignty is worth reclaiming again, and that AI memory is the highest-stakes place to reclaim it – motivates the effort; what this paper contributes on top of that argument is a precise, source-grounded account of exactly which parts of the HCP-MCP stack remain to be built, and exactly which institutions have already built the rest.

References

- [1] Shah, A. V., South, T., Evans, T., Kirk, H. R., Trask, A., Weyl, E. G., & Bakker, M. A. (2025). *Robust AI Personalization Will Require a Human Context Protocol*. https://miba.dev/assets/publications/HCP_ArXiv_2025.pdf
- [2] Shah, A. V., South, T., Evans, T., Kirk, H. R., Pei, J., Trask, A., Weyl, E. G., & Bakker, M. A. (2026). *Robust AI Personalization Controls: The Human Context Protocol*. Stanford Digital Economy Lab. <https://digitaleconomy.stanford.edu/publication/robust-ai-personalization-controls-the-human-context-protocol/>
- [3] Pentland, S., South, T., Greenwood, D., Pei, J., He, Z., & Moskowitz, B. (2026). *The Future of Personal AI: Portable and Persistent Personal Memory through a Unified Human Context Protocol*. Global Memory Workshop Final Report, Stanford HAI / Gates Foundation / AWS / Mozilla Foundation / Slalom. <https://digitaleconomy.stanford.edu/publication/the-future-of-personal-ai-portable-and-persistent-personal-memory-through-a-unified-human-context-protocol/>
- [4] South, T., et al. *HCP Demo*. GitHub. <https://github.com/tobinsouth/hcp-demo>
- [5] Packer, C., et al. (2023). *MemGPT: Towards LLMs as Operating Systems*. arXiv:2310.08560. <https://arxiv.org/abs/2310.08560>
- [6] Chhikara, P., et al. (2025). *Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory*. arXiv:2504.19413. <https://arxiv.org/abs/2504.19413>
- [7] Anthropic (2025). *Effective Context Engineering for AI Agents*. <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- [8] Park, J. S., et al. (2023). *Generative Agents: Interactive Simulacra of Human Behavior*. arXiv:2304.03442. <https://arxiv.org/abs/2304.03442>
- [9] Rasmussen, P., et al. (2025). *Zep: A Temporal Knowledge Graph Architecture for Agent Memory*. arXiv:2501.13956. <https://arxiv.org/abs/2501.13956>
- [10] Zep. *Research Benchmarks*. <https://www.getzep.com/research/>
- [11] Vectorize (2026). *Mem0 vs Zep: AI Agent Memory Compared*. <https://vectorize.io/articles/mem0-vs-zep>
- [12] BBC News (2024). *Tim Berners-Lee: I invented the web. Here’s how we take it back*. <https://www.bbc.com/news/business-68286395>
- [13] Solid Project. *About Solid*. <https://solidproject.org/about>
- [14] Inrupt. *Key Concepts*. <https://docs.inrupt.com/getting-started/key-concepts>
- [15] Schneier, B. (2020). *Inrupt: Tim Berners-Lee’s Solid*. Schneier on Security. https://www.schneier.com/blog/archives/2020/02/inrupt_tim_bern.html

- [16] Saad-Falcon, J., Narayan, A., Manihani, R., Bhathal, T., Shandilya, H., Akengin, H. O., Bo, G., Park, A., Hart, M., Costello, C., Li, C., Ré, C., & Mirhoseini, A. (2026). *OpenJarvis: Personal AI, On Personal Devices*. Stanford Hazy Research / Scaling Intelligence Lab. arXiv:2605.17172. <https://arxiv.org/abs/2605.17172>
- [17] Apple Support. *Apple Intelligence and Privacy*. <https://support.apple.com/guide/iphone/apple-intelligence-and-privacy-iphe3f499e0e/ios>
- [18] Apple (2024). *Platforms State of the Union – WWDC24*. <https://developer.apple.com/videos/play/wwdc2024/102/>
- [19] Dittmar, Y., Stephan, M. J., Völkl, T., Hollick, M., & Classen, J. (2026). *Unlocking Apple’s Private Cloud Compute: An Analysis of Privacy-Preserving Artificial Intelligence*. arXiv:2605.24239. <https://arxiv.org/abs/2605.24239>
- [20] Model Context Protocol Contributors. *Knowledge Graph Memory Server*. GitHub, [modelcontextprotocol/servers](https://github.com/modelcontextprotocol/servers). <https://github.com/modelcontextprotocol/servers/tree/main/src/memory>
- [21] Khemani, S. *How Gemini’s Memory Works*. <https://www.shloked.com/writing/gemini-memory>
- [22] OpenAI Help Center. *How do I view or delete a memory?* <https://help.openai.com/en/articles/8983145-how-do-i-view-or-delete-a-memory>
- [23] Anthropic. *Use Claude’s chat search and memory to build on previous context*. <https://support.anthropic.com/en/articles/11817273-using-claude-s-chat-search-and-memory-to-build-on-previous-context>
- [24] Anthropic. *Import and export your memory from Claude*. <https://support.anthropic.com/en/articles/12123587-import-and-export-your-memory-from-claude>
- [25] Google. *Temporary Chats and Privacy Controls in Gemini*. <https://blog.google/products-and-platforms/products/gemini/temporary-chats-privacy-controls/>
- [26] Microsoft Support. *Retrace Your Steps with Recall*. <https://support.microsoft.com/en-us/windows/ai/ai-features/retrace-your-steps-with-recall>
- [27] Perplexity Help Center. *Memory*. <https://www.perplexity.ai/help-center/en/articles/10968016-memory>
- [28] Anthropic (2024). *Introducing the Model Context Protocol*. <https://www.anthropic.com/news/model-context-protocol>
- [29] SAP Insiders. *Model Context Protocol (MCP): A Deep Dive*. <https://sapinsiders.org/posts/model-context-protocol-mcp-deep-dive>
- [30] TechCrunch (2025). *OpenAI adopts rival Anthropic’s standard for connecting AI models to data*. <https://techcrunch.com/2025/03/26/openai-adopts-rival-anthropics-standard-for-connecting-ai-models-to-data/>
- [31] Google (2025). *Gemini 2.5: Our most intelligent models are getting even better*. <https://blog.google/innovation-and-ai/models-and-research/google-deepmind/google-gemini-updates-io-2025/>
- [32] Google Developers Blog (2025). *A2A: A New Era of Agent Interoperability*. <https://developers.googleblog.com/en/a2a-a-new-era-of-agent-interoperability/>
- [33] Google Cloud Blog. *Agent2Agent Protocol Is Getting an Upgrade*. <https://cloud.google.com/blog/products/ai-machine-learning/agent2agent-protocol-is-getting-an-upgrade>
- [34] A2A Project. *Agent2Agent Protocol*. GitHub. <https://github.com/a2aproject/A2A>
- [35] Tech Policy Press. *The Right to Be Forgotten Is Dead: Data Lives Forever in AI*. <https://techpolicy.press/the-right-to-be-forgotten-is-dead-data-lives-forever-in-ai>
- [36] European Data Protection Board (2025). *AI: Effective Implementation of Data Subjects’ Rights*. https://www.edpb.europa.eu/system/files/2025-01/d2-ai-effective-implementation-of-data-subjects-rights_en.pdf
- [37] Wang-Tomic, L., Johnson, G. (2025). *New Bulk Data Operations: Update, Delete, and Fetch by Metadata*. Pinecone. <https://www.pinecone.io/blog/update-delete-and-fetch-by-metadata/>

- [38] Chakrabortii, C., Alvarado, J.G., Abdulofizova, S., Dwivedi, S. (2026). *Ghost Vectors: Soft-Deleted Embeddings Remain Reconstructible in HNSW Vector Databases*. arXiv:2606.18497. <https://arxiv.org/abs/2606.18497>
- [39] Dong, Y., et al. (2025). *Memory Injection Attacks on LLM Agents via Query-Only Interaction*. arXiv:2503.03704. <https://arxiv.org/abs/2503.03704>
- [40] OWASP. *MCP Tool Poisoning*. https://owasp.org/www-community/attacks/MCP_Tool_Poisoning
- [41] Narang, S. (2025). *FAQ: CVE-2025-54135 and CVE-2025-54136 Vulnerabilities in Cursor (CurXecute and MCPoison)*. Tenable. <https://www.tenable.com/blog/faq-cve-2025-54135-cve-2025-54136-vulnerabilities-in-cursor-curxecute-mcpoison>
- [42] JFrog (2025). *MCP Prompt Hijacking Vulnerability*. <https://jfrog.com/blog/mcp-prompt-hijacking-vulnerability/>
- [43] Invariant Labs. *MCP Security Notification: Tool Poisoning Attacks*. <https://invariantlabs.ai/blog/mcp-security-notification-tool-poisoning-attacks>